

산업재해 예방을 위한
음성기반
우울증 예측 서비스



INDEX



서규현

총괄 및 기획



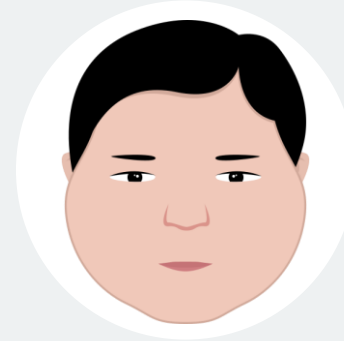
김대운

기획



정우성

개발



강경진

디자인

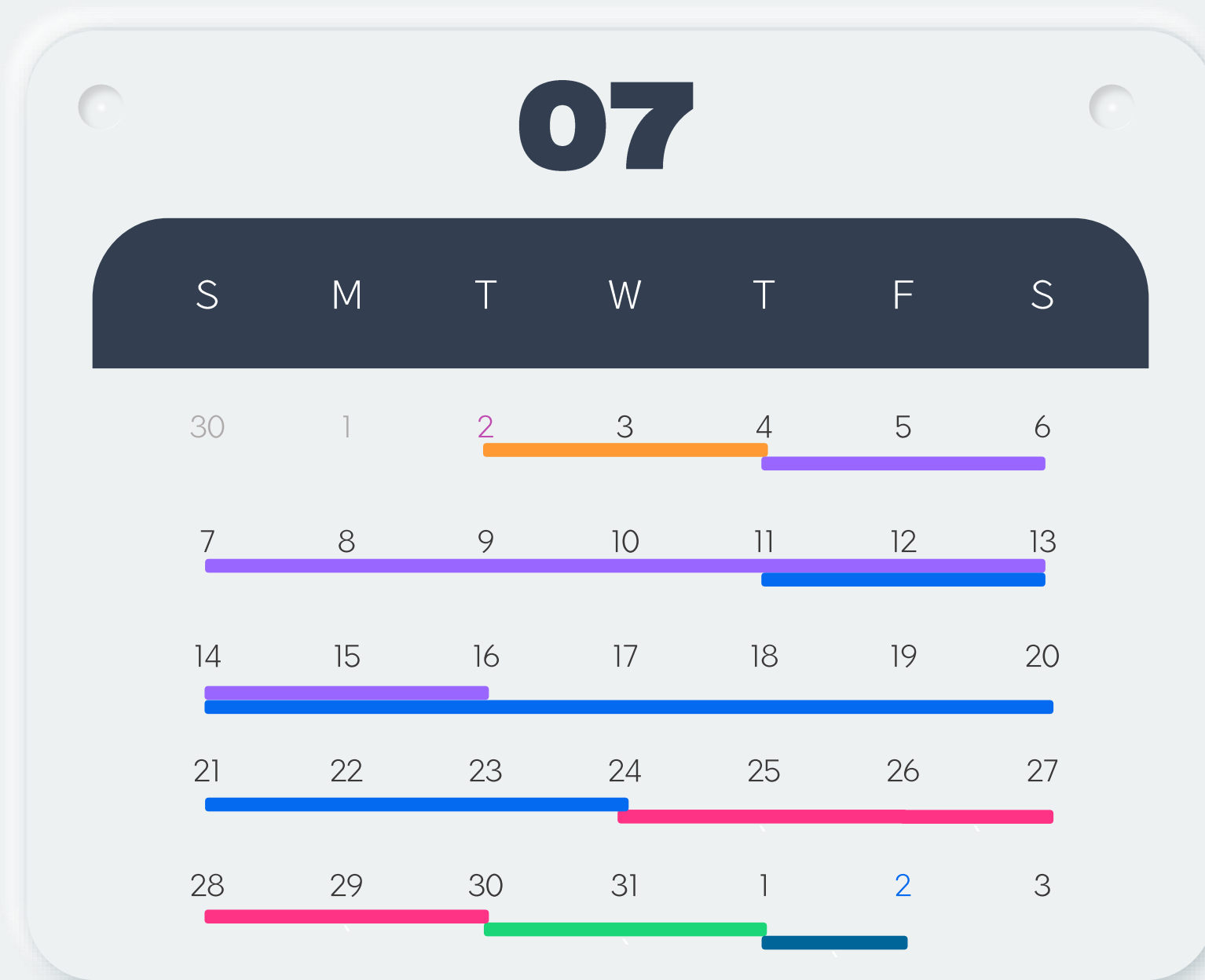


정혜주

디자인

프로젝트 캘린더

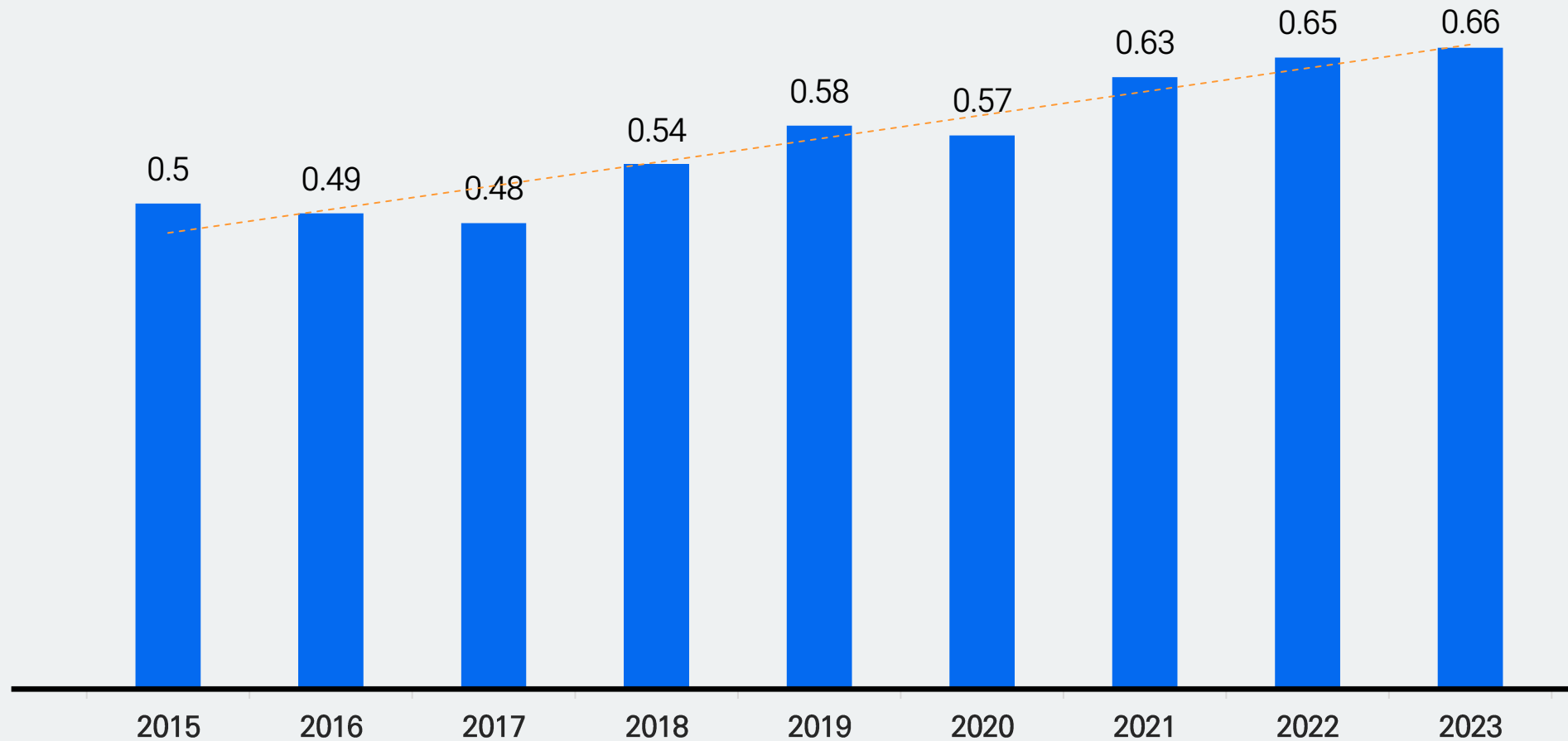
- 주제선정
- 기획
- 개발
- PT디자인
- 발표준비
- 발표



INDEX

1. 문제 배경
2. 문제 도출
3. 솔루션
 - 서비스 소개
 - 시스템 아키텍처
 - 서비스 아키텍처
 - 서비스 활용 방안
4. 시장조사
5. 기대효과
6. 로드맵

한국의 산업재해는 증가세를 보임



산업재해의 발생원인은
물적요인과 인적요인으로 구분

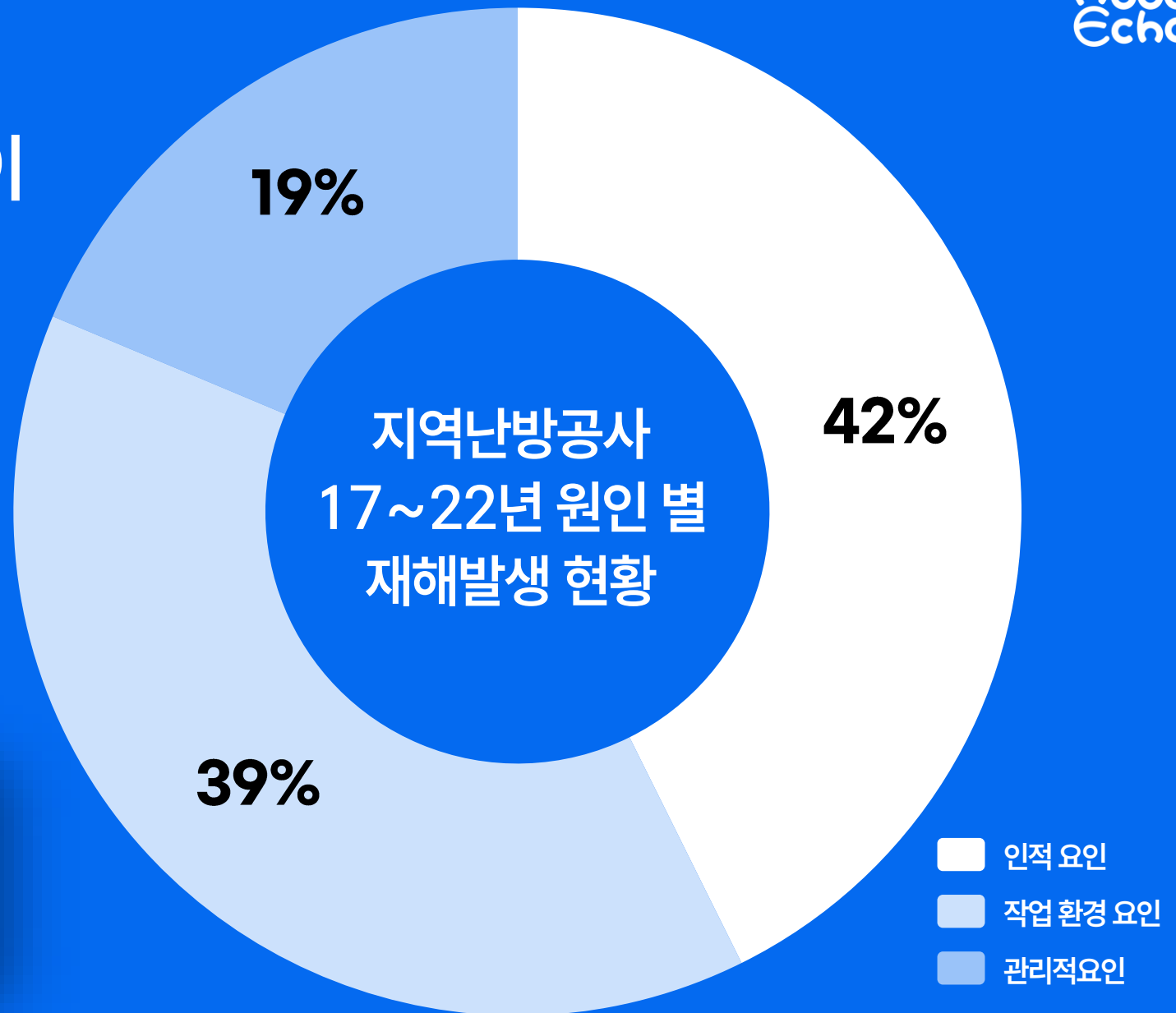
산업재해

물적요인

인적요인

문제 배경

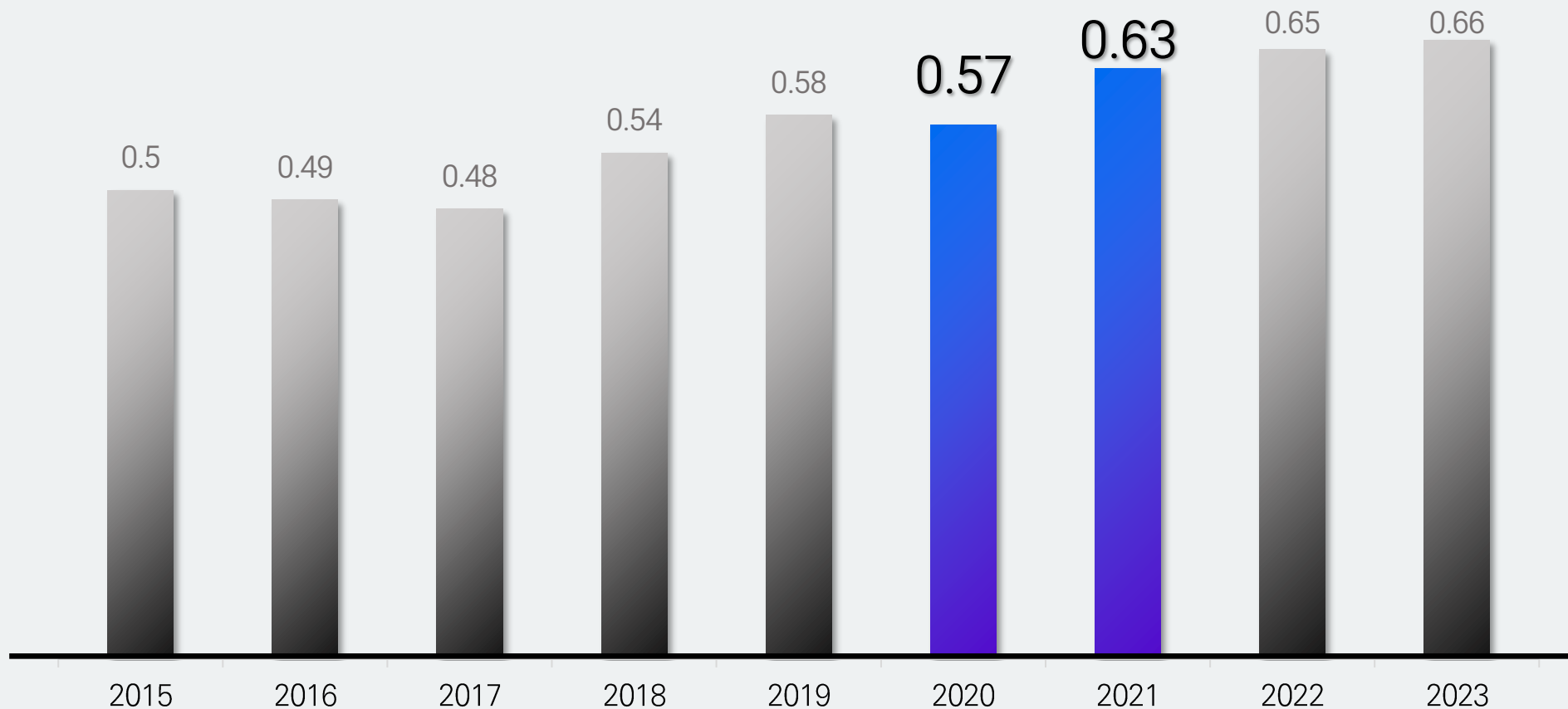
인적요인으로 발생하는 비율이
42%로 큰 비중을 차지



인적요인 - 작업자 부주의, 불안전한 행동 등
 작업 환경 요인 - 안전기준 및 작업기준 미준수 등
 관리적 요인 - 관리조직의 결함, 교육 부재 등

2020년과 2021년, 산업재해 발생이 큰 증가폭을 보임

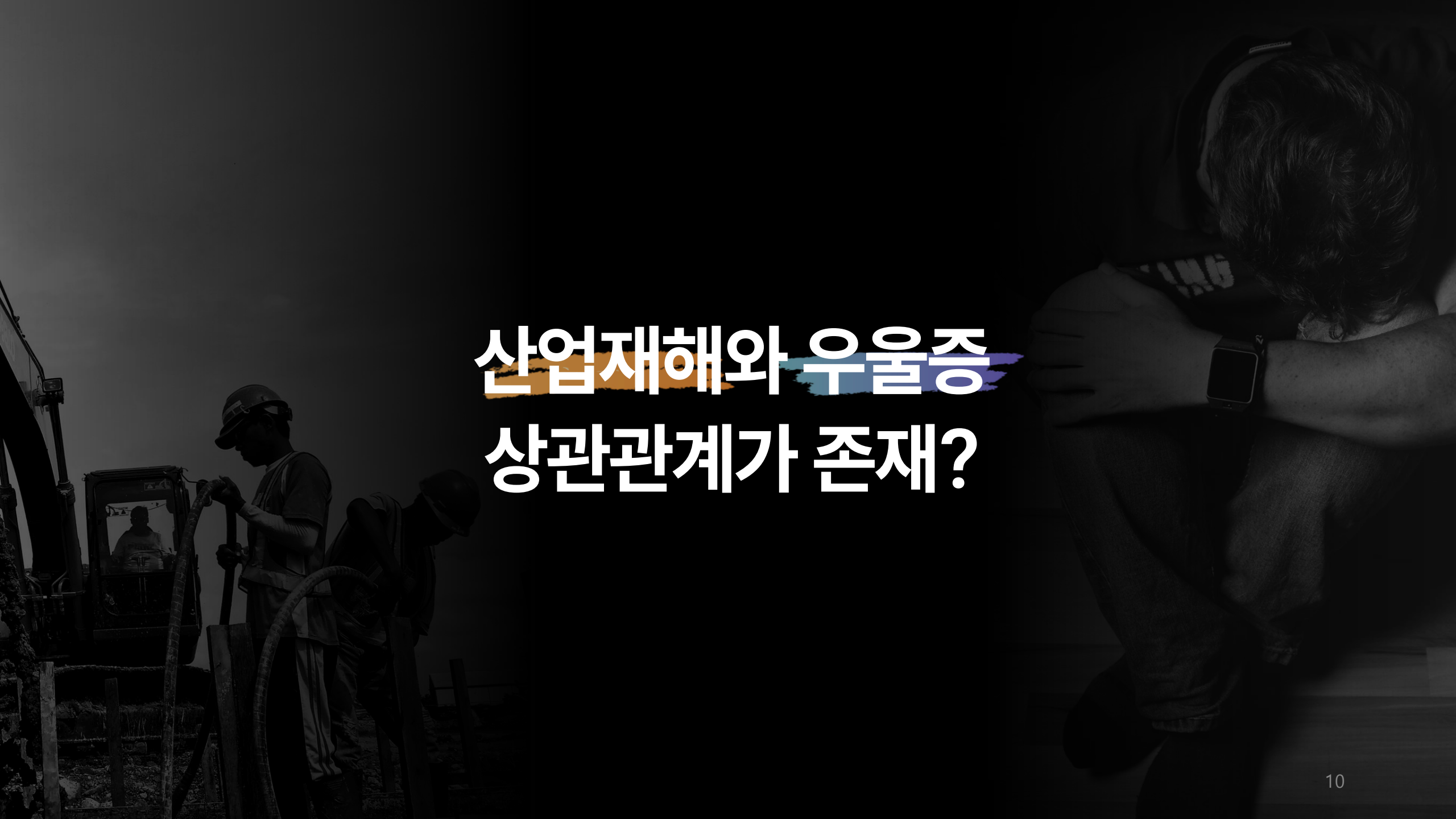
출처 : 고용노동부, 「산업재해 현황분석」



CORONA BLUE

코로나블루(Corona Blue)

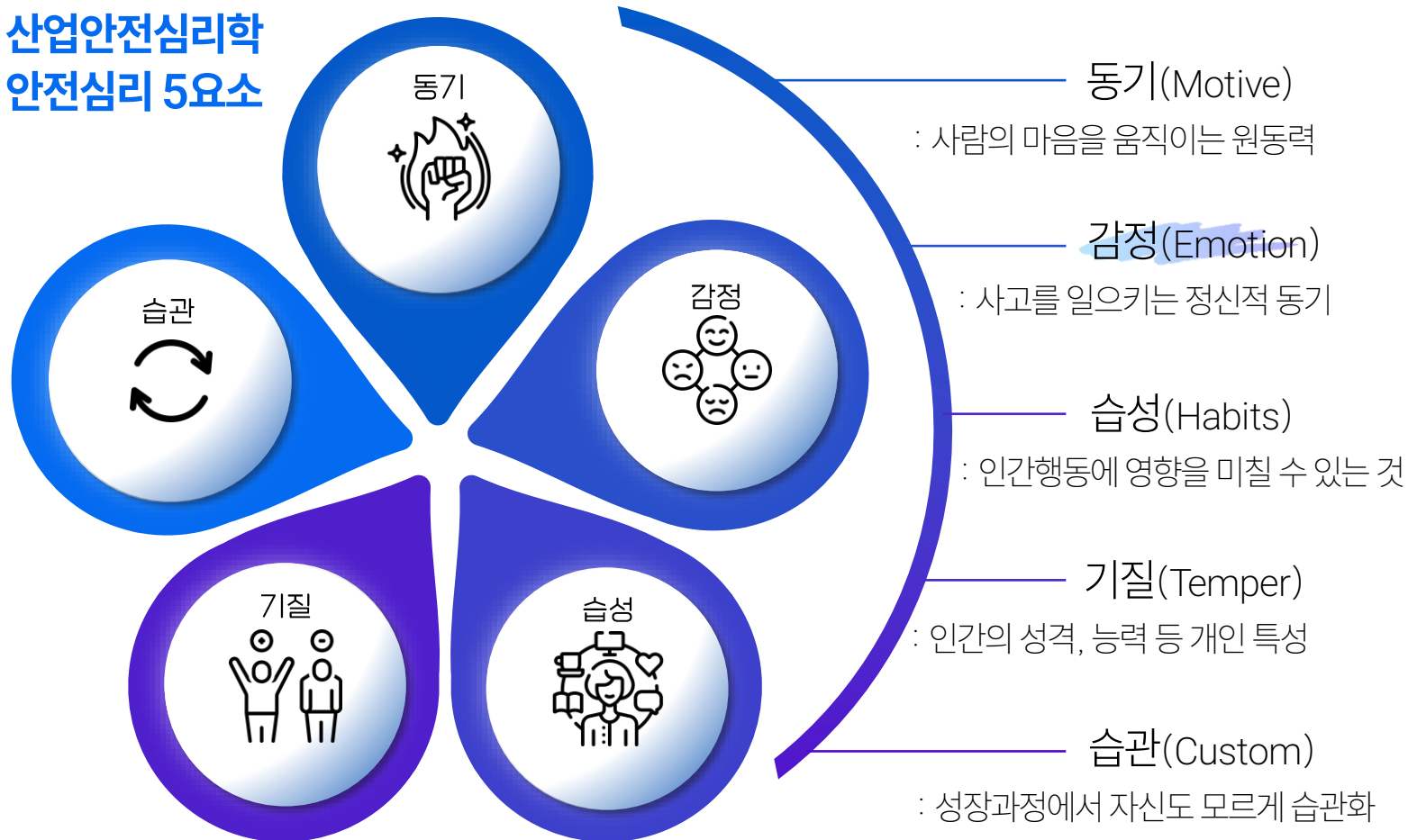
코로나19와 우울감(Blue)이 합쳐진 신조어.
코로나19 확산으로 일상에 큰 변화를 겪으면서 생긴
우울감이나 무기력증을 뜻한다.



산업재해와 우울증 상관관계가 존재?

산업안전심리학 교재나 논문에서 우울과 산업재해의 관련성을 보여줌.

산업안전심리학 안전심리 5요소



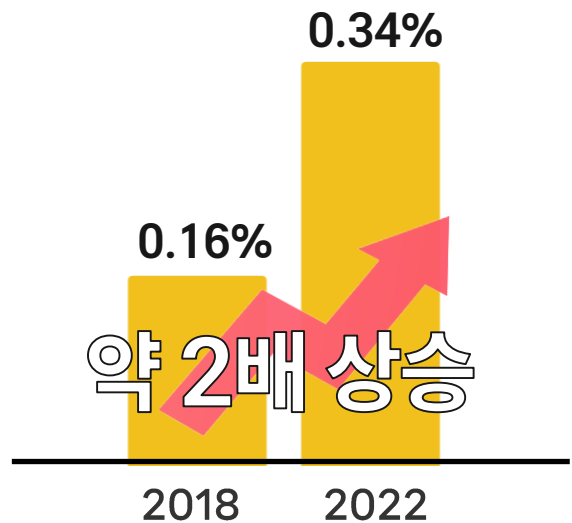
철도 기관사의 사고와 신체적 심리적 우울



산재 근로자 중 정신질환 비율과 전체 재해율이 같이 증가하는 추세를 보임

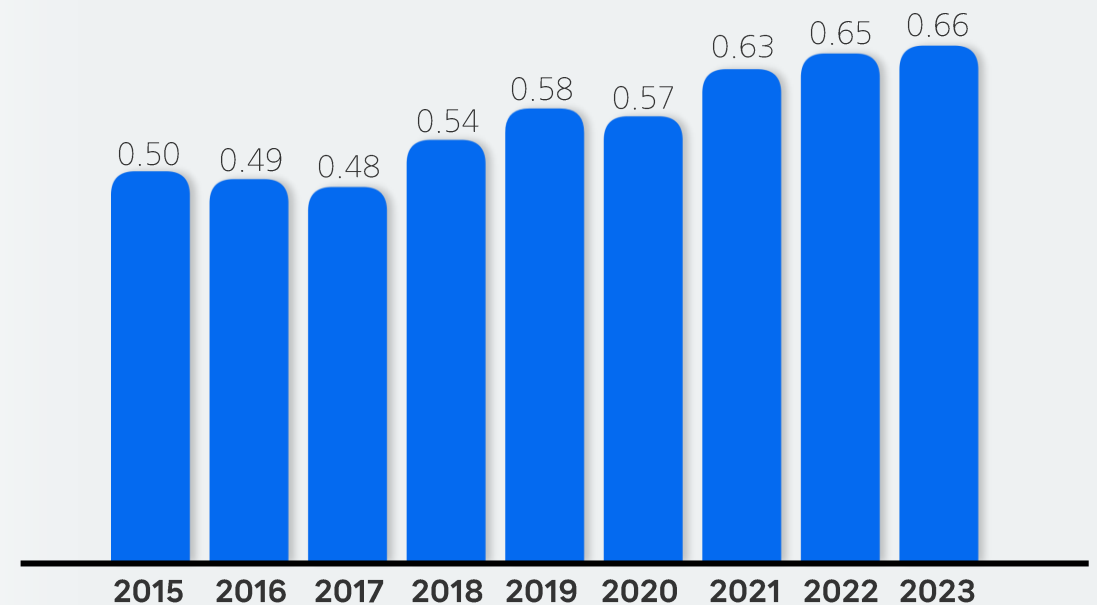
[산업재해 현황 - 제조업부문]

산재 근로자 정신질환 비율



[산업재해 현황 - 전체 재해율]

단위 : 명, %



근로자 지원 프로그램(EAP)

- 효과 측정의 어려움
- 비밀유지의 어려움
- 근로자가 서비스 존재를 불가지

* 근로자들의 직무나 생산성에 부정적인 영향을 스스로 해결하기 위해 시행하는 상담서비스

국가건강검진(PHQ-9)

- 국가건강검진에서 10년마다 시행
- 자가보고형 형식이기에 한계점 존재

*자기보고식 우울 검사



근로자 지원 프로그램(EAP)

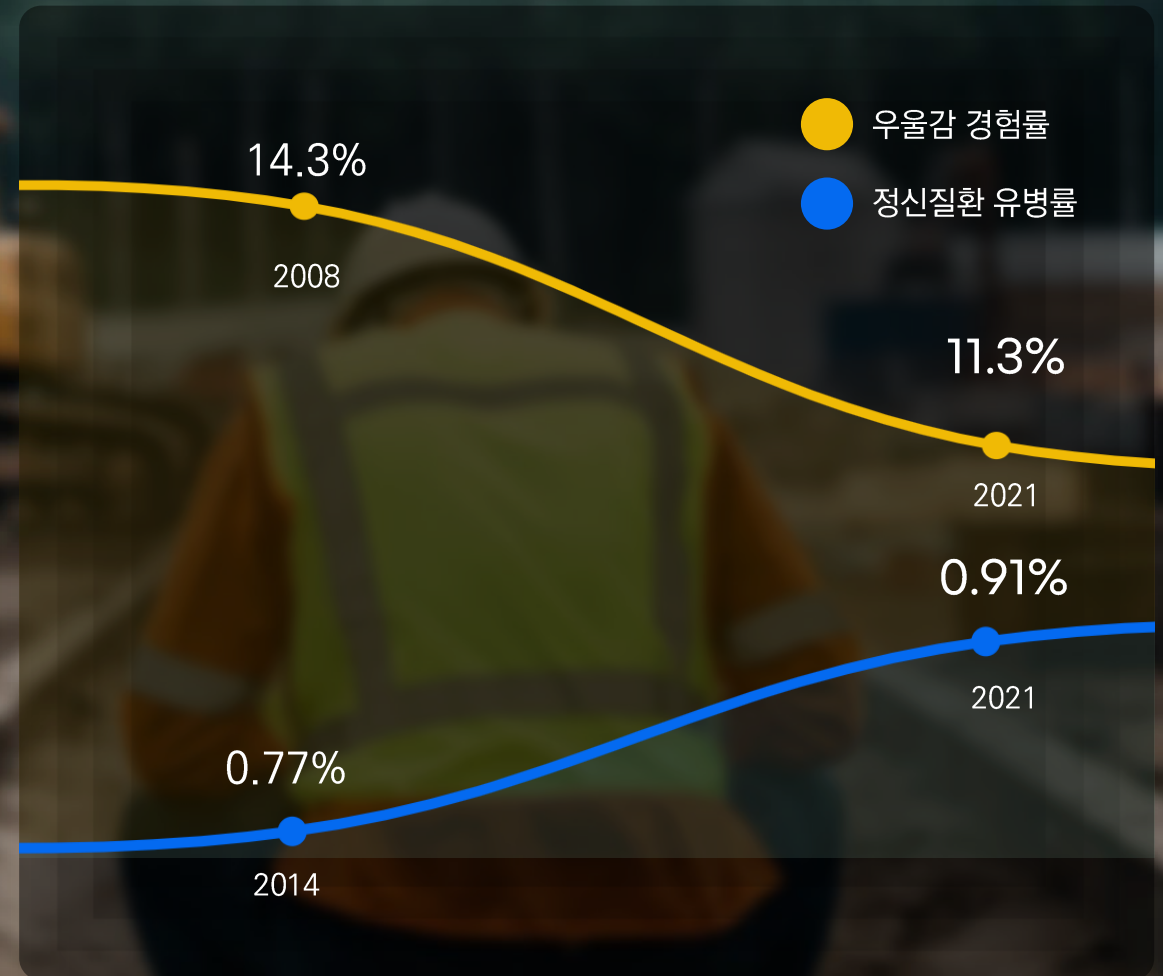
- 효과 측정의 어려움
- 비밀유지의 어려움
- 근로자가 서비스 존재를 불가지

* 근로자들의 직무나 생산성에 부정적인 영향을 스스로 해결하기 위해 시행하는 상담서비스

국가건강검진(PHQ-9)

- 국가건강검진에서 10년마다 시행
- 자가보고형 형식이기에 한계점 존재

*자기보고식 우울 검사





MoodEcho란?

음성의 반향을 통해 감정을 살핀다는 의미를 가진 Mood Echo는
근로자의 음성을 통해 초기에 쉽게 우울증 진단이 가능한 서비스이다.
음성인식을 활용하기에, 스스로 인지 못한 우울감도 선별 가능하다.

서비스 소개

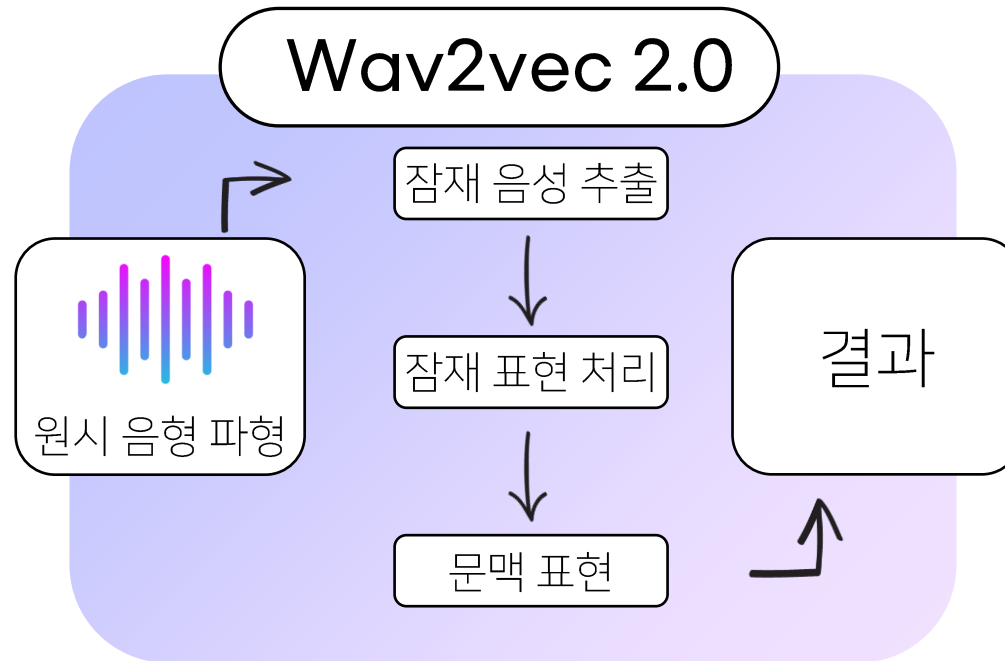
언제, 어디서나
프라이빗하게
당신의 목소리로



Depression recognition using voice-based pre-training model



DAIC-WOZ Dataset



이진 분류

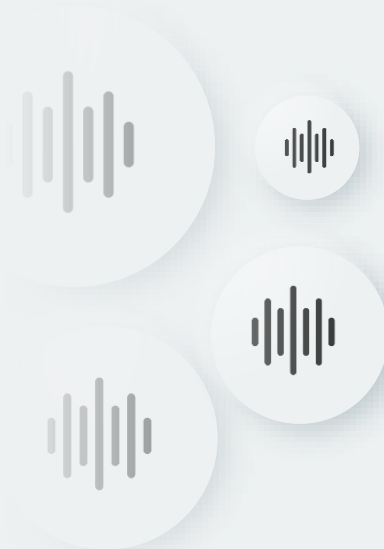
정확도 96.49%
RMSE 0.1875

다중 클래스 분류

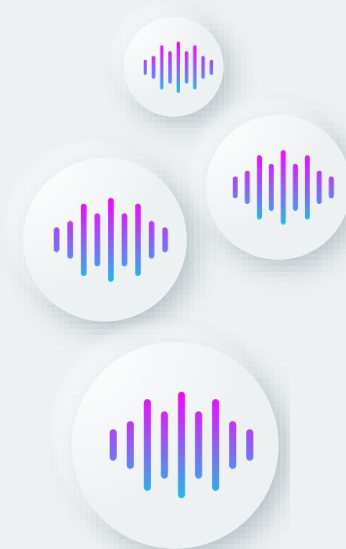
정확도 94.81%
RMSE 0.3810



DAIC-WOZ Dataset



Binary preprocessing



Mood Echo



Pre-training, Fine-tuning

```
1 class AudioDataset(Dataset):
2     def __init__(self, directory):
3         self.directory = directory
4         self.filenamees = [os.path.join(directory, f) for f in os.listdir(directory) if f.endswith('.wav')]
5         self.files = []
6
7         for file in tqdm(self.filenamees, desc="Loading audio files"):
8             audio, _ = librosa.load(file, sr=16000, mono=True)
9             self.files.append(torch.tensor(audio, dtype=torch.float32))
10
11     def __len__(self):
12         return len(self.files)
13
14     def __getitem__(self, idx):
15         return self.files[idx]
16
17 # 데이터셋 인스턴스화
18 dataset = AudioDataset('./data/reduce/train')
19 loader = DataLoader(dataset, batch_size=1, shuffle=True)
20
21 # 사전 학습된 모델 로드
22 model = Wav2Vec2ForPreTraining.from_pretrained("kresnik/wav2vec2-large-xlsr-korean")
23
24 # Trainer 설정
25 training_args = TrainingArguments(
26     output_dir='./wav2vec2_pretrained',
27     per_device_train_batch_size=1, # GPU 메모리에 따라 조정
28     num_train_epochs=10, # 적절한 에폭 수 설정
29     logging_dir='./logs',
30     logging_steps=10,
31     save_steps=500,
32     do_train=True
33 )
34
35 trainer = Trainer(
36     model=model,
37     args=training_args,
38     data_collator=lambda data: {'input_values': torch.cat([x.unsqueeze(0) for x in data], 0)},
39     train_dataset=dataset
40 )
41
42 # Pre-training 시작
43 trainer.train()
```

```
1 training_args = TrainingArguments(
2     output_dir="./results",
3     per_device_train_batch_size=1, # CPU에서는 작은 배치 크기 사용
4     gradient_accumulation_steps=8,
5     evaluation_strategy="no", # 에폭마다 평가
6     num_train_epochs=3,
7     save_strategy="epoch",
8     logging_dir='./logs',
9     learning_rate=1e-4,
10    report_to='none', # --report_to에 대한 경고에 맞춰 업데이트
11    no_cuda=True, # CUDA 비활성화
12 )
13
14 trainer = Trainer(
15     model=model,
16     args=training_args,
17     train_dataset=processed_dataset,
18     tokenizer=processor.feature_extractor
19 )
20
21 # 불필요한 변수 삭제 및 메모리 정리
22 del data, participant_labels, wav_files, data_items, dataset, processed_dataset
23
24 trainer.train()
```

458302464 bytes. Buy new RAM!

Feature 추출

Code
for
Machine Learning



```
1 def extract_features(audio_path, target_length):
2     """오디오 파일에서 특성 추출"""
3     # 오디오 파일 로드
4     speech_array, sampling_rate = torchaudio.load(audio_path)
5
6
7     # 패딩 또는 잘라내기 적용
8     speech_array = pad_or_truncate(speech_array, target_length)
9
10    # 차원을 (배치, 채널, 샘플) 형식으로 변경
11    speech_array = speech_array.unsqueeze(0)
12
13    # 음성 특징 벡터 추출 (Wav2Vec2)
14    inputs = processor(speech_array.squeeze(), sampling_rate=sampling_rate, return_tensors="pt", padding=True)
15    with torch.no_grad():
16        outputs = model(inputs.input_values)
17        wav2vec_features = outputs.last_hidden_state if hasattr(outputs, 'last_hidden_state') else outputs.hidden_states[-1]
18
19    # Mel-spectrogram 추출
20    mel_spec_transform = T.MelSpectrogram(sample_rate=sampling_rate, n_mels=64)
21    mel_spec = mel_spec_transform(speech_array).squeeze().numpy()
22
23    # MFCC 추출
24    mfcc_transform = T.MFCC(sample_rate=sampling_rate, n_mfcc=13)
25    mfcc = mfcc_transform(speech_array).squeeze().numpy()
26
27    # Zero-Crossing Rate 추출
28    zcr_transform = T.ComputeDeltas()
29    zcr = (speech_array.squeeze().numpy() > 0).sum() / float(len(speech_array.squeeze().numpy()))
30
31    # RMS Energy 추출
32    rms_transform = T.AmplitudeToDB()
33    rms = rms_transform(speech_array).squeeze().numpy().mean()
34
35    return wav2vec_features.squeeze().numpy(), mel_spec, mfcc, zcr, rms
```

Feature result

Code
for
Machine Learning

	0	1	2	3	4	...	843	844	845	846
0	-0.032061	0.005627	-0.050635	-0.011428	-0.116650	...	-4.793350	-11.319645	0.320375	-72.627251
1	-0.024550	0.028865	-0.055667	-0.047210	0.084196	...	-5.023795	-14.309443	0.469125	-65.792953
2	-0.034020	-0.003702	0.027991	-0.073814	-0.006831	...	-10.622462	-10.210487	0.439625	-70.094482
3	-0.017872	0.021592	-0.060968	-0.062977	0.083948	...	-13.637566	-7.170184	0.482125	-66.477318
4	-0.002252	0.020875	-0.111379	-0.046847	-0.018558	...	-13.442705	-9.371253	0.433375	-71.782356
...	...	...	...	...	...	...	...	...	...	...
4868	-0.024757	0.025958	-0.083393	-0.060649	0.031224	...	-16.761213	-8.503918	0.430312	-70.218102
4869	-0.008153	0.019904	-0.129245	-0.051861	0.059924	...	-16.260597	-8.020310	0.353875	-76.023338
4870	-0.038740	-0.003638	-0.150115	-0.059391	0.030270	...	-10.000116	-3.646889	0.461125	-68.183830
4871	-0.009823	0.029765	-0.076572	-0.043462	0.049649	...	-5.426978	-9.824192	0.469750	-67.822655
4872	-0.116351	-0.003604	-0.028579	-0.079261	-0.060474	...	-15.318240	-7.088049	0.438063	-69.595001



모델 학습 및 초기 정확도

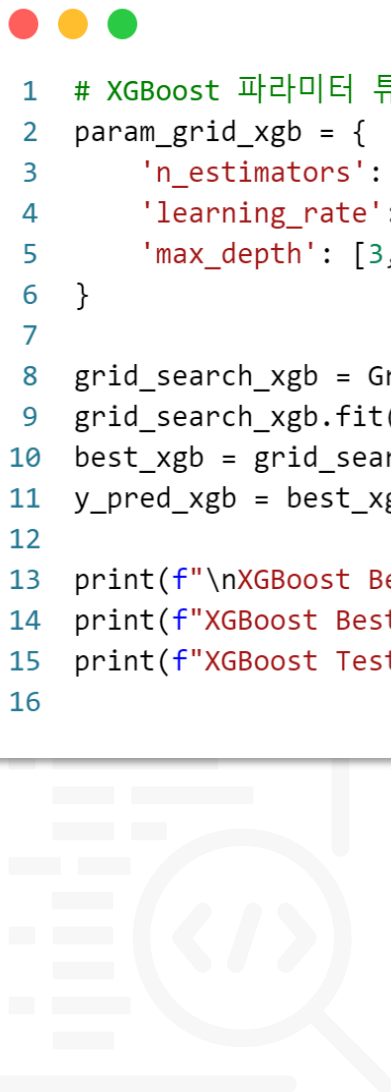
Code
for
Machine Learning

```
1 # X, y 로드
2 X = pd.read_csv('./features/base-960h/audio_features_base-960h.csv')
3 y = pd.read_csv('./features/base-960h/audio_label_binary_base-960h.csv')
4
5 # 데이터셋 분할
6 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
7
8 # 모델 학습 및 초기 정확도 측정
9 models = {
10     "Logistic Regression": LogisticRegression(max_iter=1000),
11     "Support Vector Machine": SVC(),
12     "Gradient Boosting": GradientBoostingClassifier(),
13     "XGBoost": xgb.XGBClassifier(use_label_encoder=False, eval_metric='logloss')
14 }
15
16 accuracies = {}
17
18 for name, model in models.items():
19     model.fit(X_train, y_train)
20     y_pred = model.predict(X_test)
21     accuracy = accuracy_score(y_test, y_pred)
22     accuracies[name] = accuracy
23     print(f"{name} Accuracy: {accuracy * 100:.2f}%")
```

Result

Logistic Regression Accuracy: 70.59%
Support Vector Machine Accuracy: 72.03%
Gradient Boosting Accuracy: 73.67%
XGBoost Accuracy: 70.80%

파라미터 튜닝 및 결과값



```
1 # XGBoost 파라미터 튜닝
2 param_grid_xgb = {
3     'n_estimators': [100, 200, 300],
4     'learning_rate': [0.01, 0.1, 0.2],
5     'max_depth': [3, 4, 5]
6 }
7
8 grid_search_xgb = GridSearchCV(xgb.XGBClassifier(use_label_encoder=False, eval_metric='logloss'), param_grid_xgb, cv=5, scoring='accuracy')
9 grid_search_xgb.fit(X_train, y_train)
10 best_xgb = grid_search_xgb.best_estimator_
11 y_pred_xgb = best_xgb.predict(X_test)
12
13 print(f"\nXGBoost Best Parameters: {grid_search_xgb.best_params_}")
14 print(f"XGBoost Best Cross-validation Accuracy: {grid_search_xgb.best_score_*100:.2f}")
15 print(f"XGBoost Test Accuracy: {accuracy_score(y_test, y_pred_xgb)*100:.2f}")
16
```

Result

```
XGBoost Best Parameters: {'learning_rate': 0.01, 'max_depth': 5, 'n_estimators': 300}
XGBoost Best Cross-validation Accuracy: 68.83
XGBoost Test Accuracy: 72.54
```


파라미터 튜닝 및 결과값

Code
for
Machine Learning

기본 세팅(binary)

	Logistic	SVM	Gradient Boosting	XGBoost
korean	66.39	72.03	70.70	71.00
base	71.11	72.03	72.34	72.13
base-960h	70.59	72.03	73.67	70.80
xlsr-53-english	72.03	72.03	71.21	71.11
large-960h	69.57	72.03	72.13	71.21

파라 세팅(binary)

	Logistic	SVM	Gradient Boosting	XGBoost
korean	68.55	72	72.03	72.23
	C: 0.1, solver: newton-cg	C: 0.1, , gamma: 1, kernel: rbf	learning_rate: 0.01, max_depth: 3, n_estimators: 200	learning_rate 0.01, max_depth: 5, n_estimators: 300
base-960h	72.03	72.03	72.64	72.34
	C: 1, solver: newton-cg	C: 0.1, gamma: 1, kernel: rbf	learning_rate: 0.01, max_depth: 4, n_estimators: 200	learning_rate: 0.01, max_depth: 3, n_estimators: 200

K-fold 교차 검증

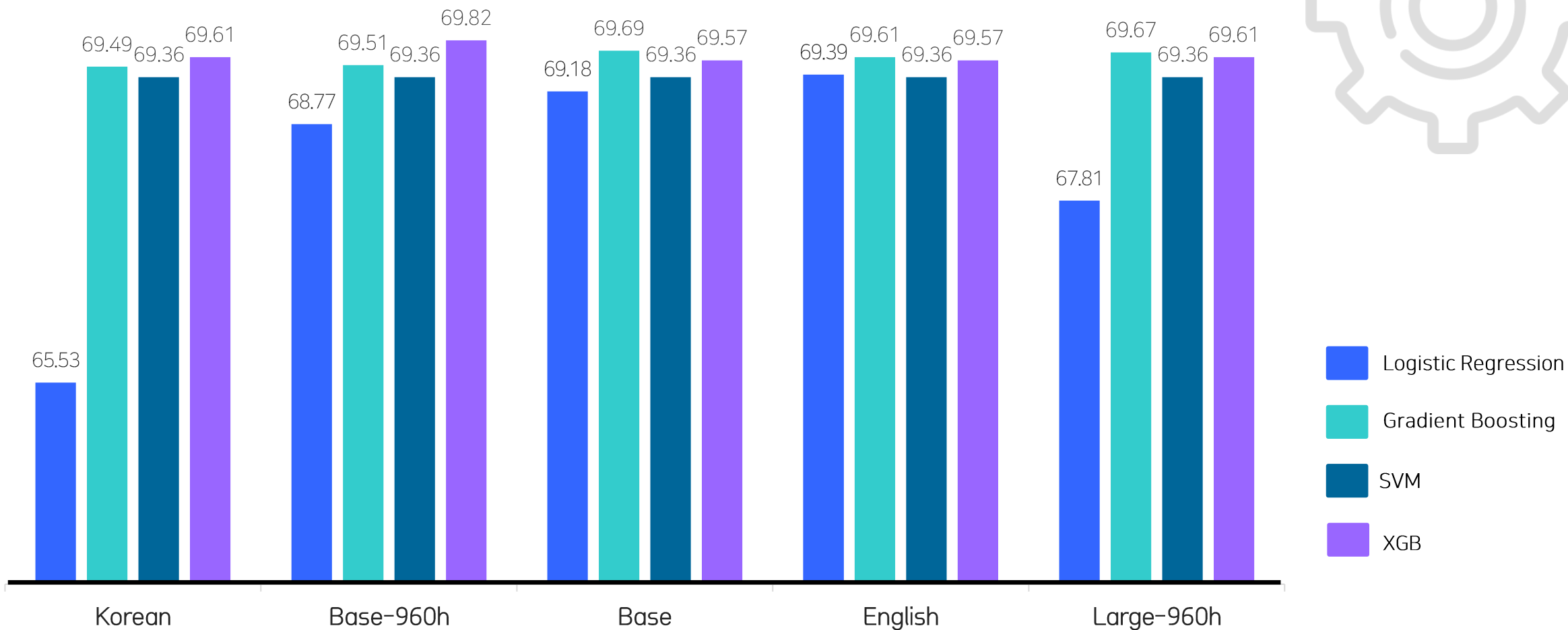
Code
for
Machine Learning



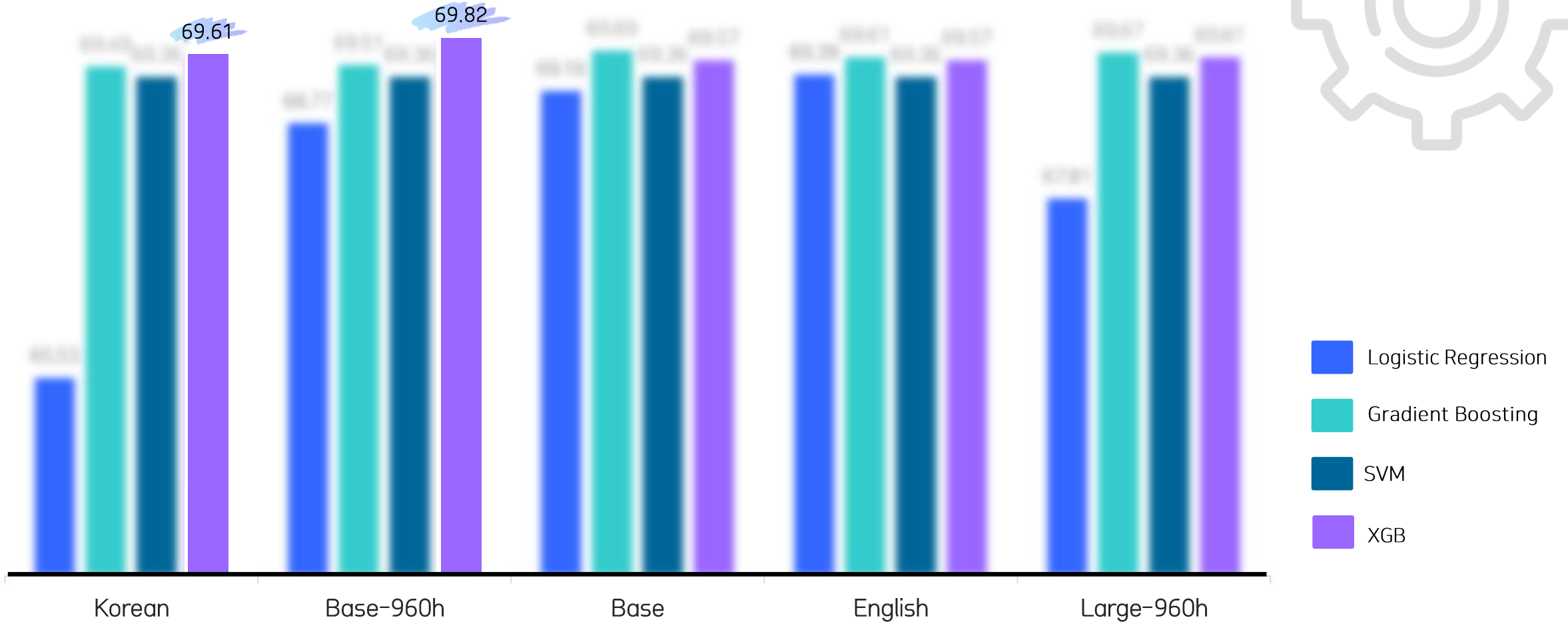
```
1 # 모델 설정
2 model = xgb.XGBClassifier(use_label_encoder=False, eval_metric='logloss', learning_rate = 0.01, max_depth = 5, n_estimators= 300)
3
4 kf = KFold(n_splits=5, shuffle=True, random_state=42)
5
6 # 교차 검증 수행 (5-폴드)
7 scores = cross_val_score(model, X, y, cv=kf, scoring='accuracy')
8
9 # 교차 검증 결과 출력
10 print('base-960h xgb')
11 print(f"각 폴드의 정확도: {scores}")
12 print(f"평균 정확도: {np.mean(scores)*100:.2f}")
```



K-fold 교차 검증



K-fold 교차 검증



XGBoost 모델 학습

Code
for
Machine Learning



```
1 # 데이터 불러오기
2 X = pd.read_csv('./features/base-960h2/audio_features_base-960h2.csv')
3 y = pd.read_csv('./features/base-960h2/audio_label_binary_base-960h2.csv')
4
5 # 데이터셋 분할
6 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
7
8 # 하이퍼파라미터 설정
9 params = {
10     'n_estimators': 300,
11     'learning_rate': 0.01,
12     'max_depth': 5
13 }
14
15 # XGBoost 모델 학습
16 model = xgb.XGBClassifier(**params, use_label_encoder=False, eval_metric='logloss')
17 model.fit(X_train, y_train)
18
19 # 모델 예측
20 y_pred = model.predict(X_test)
21
22 # 정확도
23 accuracy = accuracy_score(y_test, y_pred)
24
25 # 평가
26 print(f"Accuracy: {accuracy}")
27
28 # 모델 저장
29 model_filename = './모델/base-960h2_XGB.pkl'
30 joblib.dump(model, model_filename)
```

모델 성능 test

Code
for
Machine Learning



```
1 # ID 320 (PHQ8_score : 11)
2 for i in range(3,13):
3     result = predict_depression(f'./data/train/320_processed_{i}.wav')
4     print(result)
```

Result

```
[1][1][0][0][0]
[1][0][0][0][0]
```



```
1 # ID 343 (PHQ8_score : 9)
2 for i in range(3,13):
3     result = predict_depression(f'./data/train/343_processed_{i}.wav')
4     print(result)
```

Result

```
[0][0][0][0][0]
[0][0][0][0][0]
```



```
1 # ID 320 (PHQ8_score : 11)
2 for i in range(1,3):
3     result = predict_depression(f'./data/test/320_processed_{i}.wav')
4     print(result)
```

Result

```
[0][1]
```



모델 성능 test

Code
for
Machine Learning



train_base-960h.csv
test_base-960h.csv
train_korean.csv
test_korean.csv

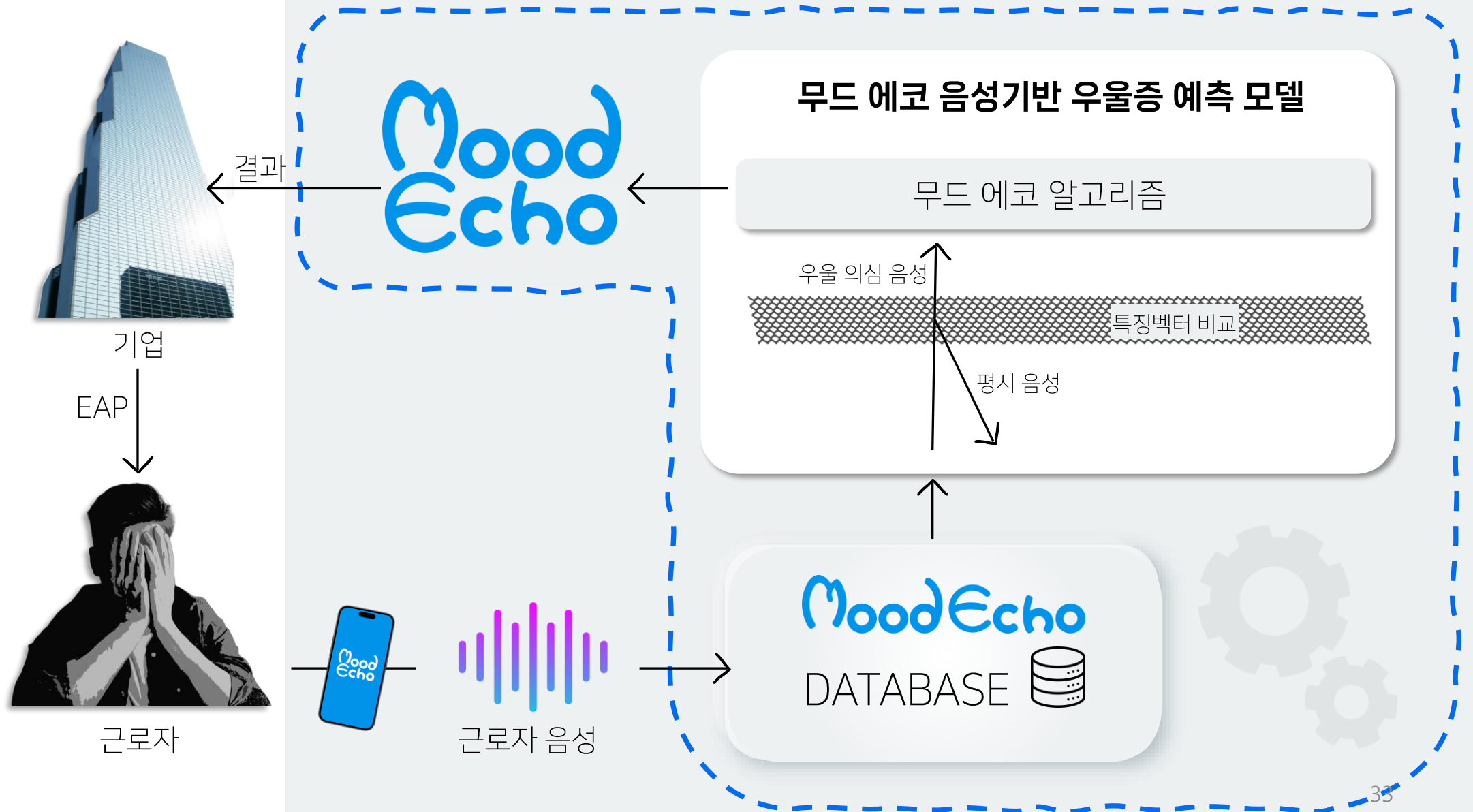
Participant_ID	PHQ8_Binary	PHQ8_Score	Predicted_PHQ8_Binary
333	0	5	[0]
336	0	7	[0]
338	1	15	[1]
339	1	11	[1]
340	0	1	[0]
341	0	7	[0]
343	0	9	[0]
344	1	11	[1]
345	1	15	[1]
347	1	16	[0]
348	1	20	[0]
350	1	11	[1]
351	1	14	[0]
352	1	10	[1]
353	1	11	[1]

모델 성능 test

Code
for
Machine Learn

```
1 merged_df = pd.read_csv('./예측/test_base-960h2.csv')
2
3 merged_df['Predicted_PHQ8_Binary'] = merged_df['Predicted_PHQ8_Binary'].apply(lambda x: x.strip('[]')).astype(int)
4
5 accuracy = accuracy_score(merged_df['PHQ8_Binary'], merged_df['Predicted_PHQ8_Binary'])
6 f1 = f1_score(merged_df['PHQ8_Binary'], merged_df['Predicted_PHQ8_Binary'])
7 precision = precision_score(merged_df['PHQ8_Binary'], merged_df['Predicted_PHQ8_Binary'])
8 recall = recall_score(merged_df['PHQ8_Binary'], merged_df['Predicted_PHQ8_Binary'])
9
10 print(f"Accuracy: {accuracy}")
11 print(f'F1 Score: {f1}')
12 print(f'Precision: {precision}')
13 print(f'Recall: {recall}')
```

Dataset	Accuracy	F1 Score	Precision	Recall
korean-train	0.968	0.944	0.981	0.911
korean-test	0.639	0.566	1.0	0.395
base-960h-train	0.952	0.920	0.912	0.928
base-960h-test	0.814	0.689	1.0	0.526



Access the Mood Echo

무드에코에 접속.

Select Language

사용언어를 선택

Record Your Voice

화면 속 문장을 읽고 녹음

Complete the Session

녹음을 제출



Access the Mood Echo

무드에코에 접속.

Select Language

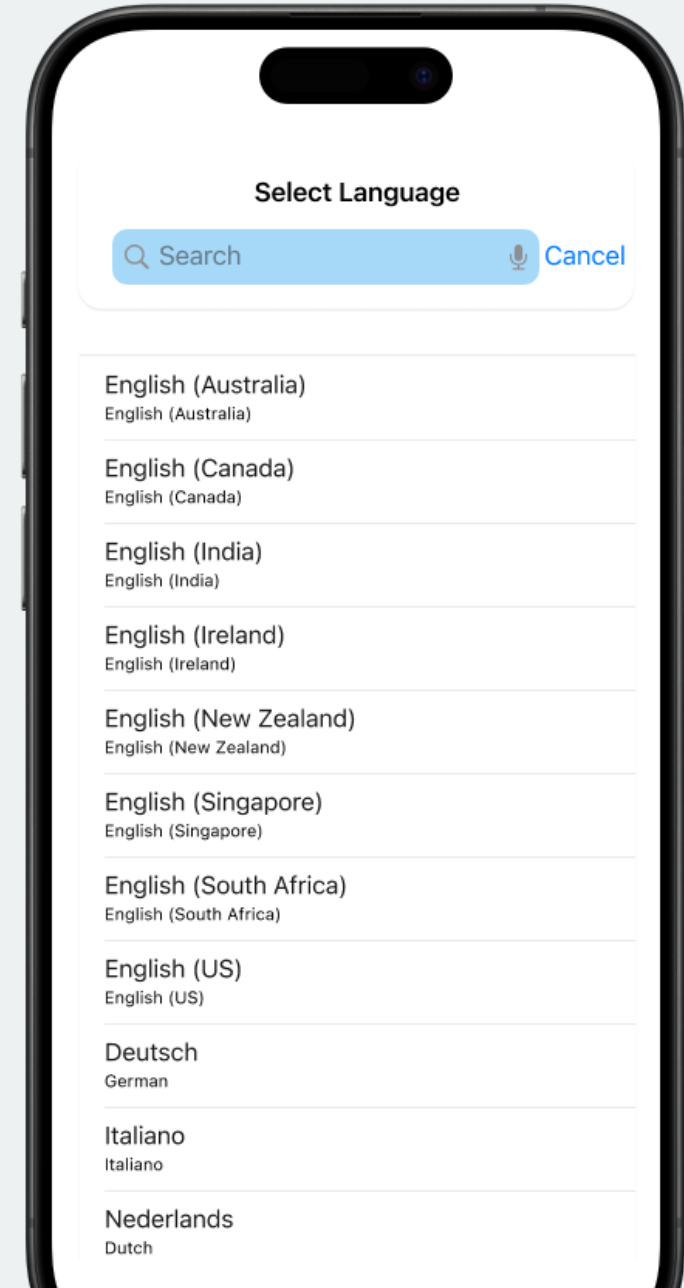
사용언어를 선택

Record Your Voice

화면 속 문장을 읽고 녹음

Complete the Session

녹음을 제출



서비스 활용 예시

Access the Mood Echo

무드에코에 접속.

Select Language

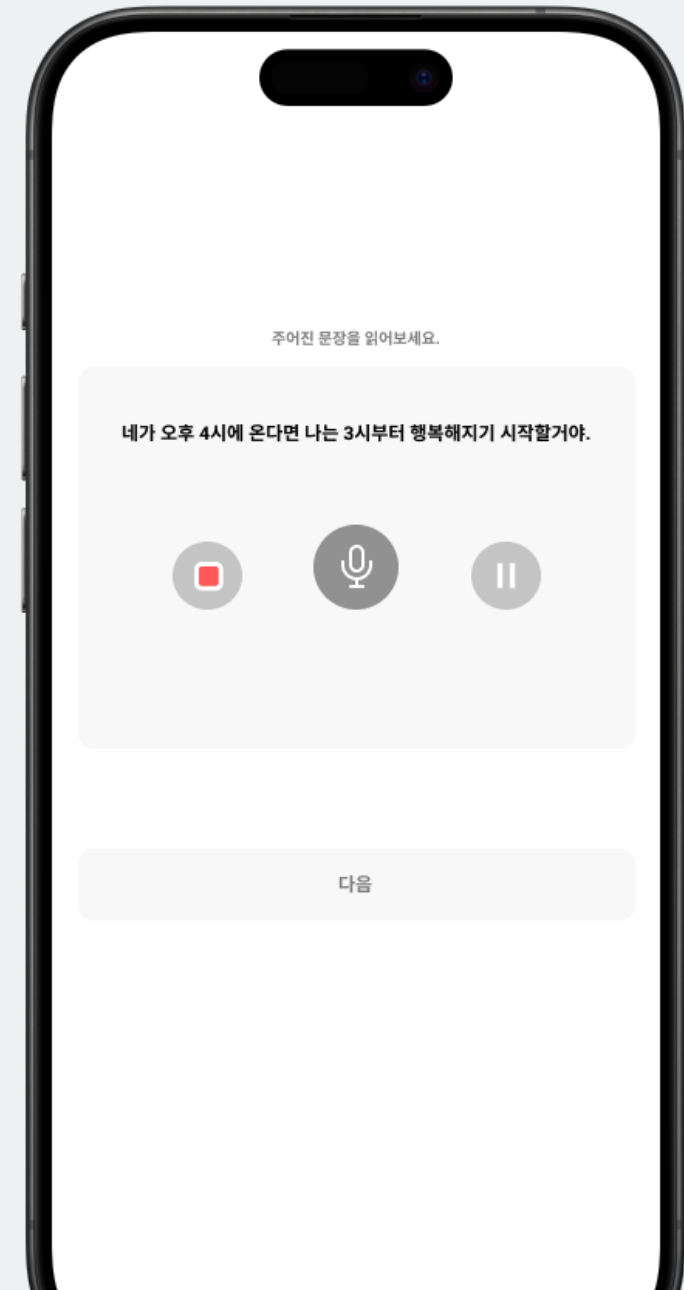
사용언어를 선택

Record Your Voice

화면 속 문장을 읽고 녹음

Complete the Session

녹음을 제출



서비스 활용 예시

Access the Mood Echo

무드에코에 접속.

Select Language

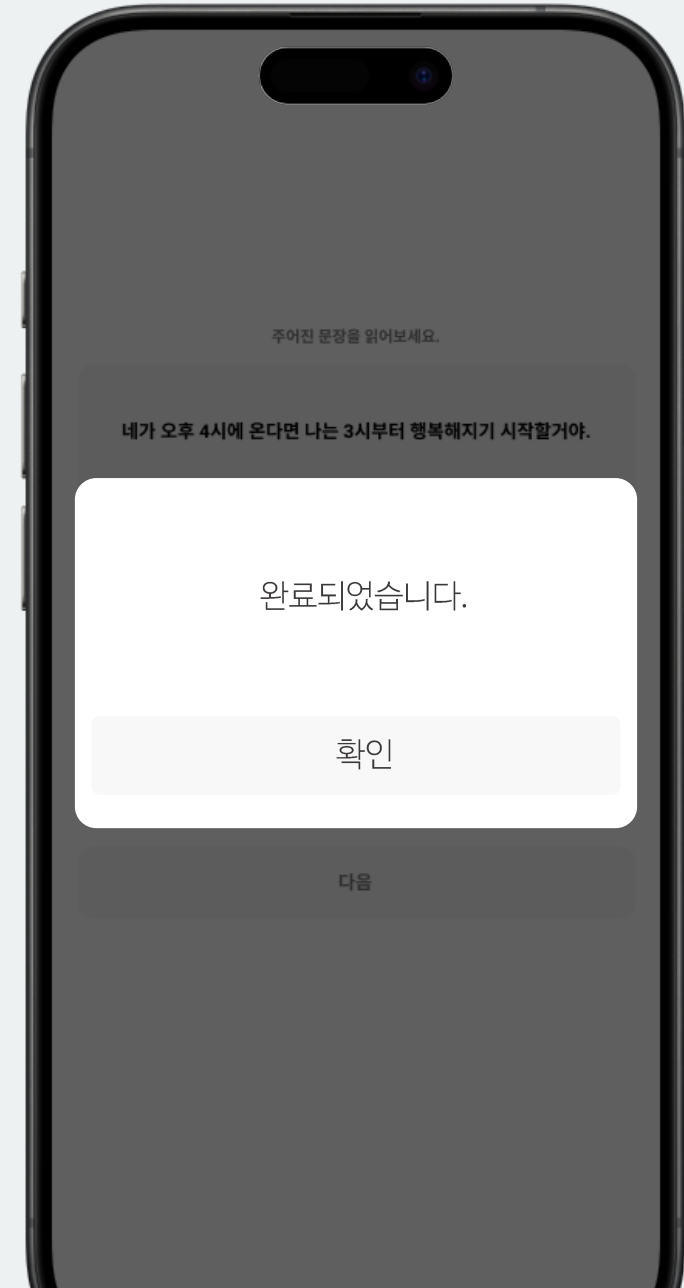
사용언어를 선택

Record Your Voice

화면 속 문장을 읽고 녹음

Complete the Session

녹음을 제출



TAM

한국의 기업 수 약 400만개
평균 직원 수 15명

전체 시장 약 6000만 명

SAM

대기업 및 중견기업 약 3만개
평균 직원 수 500명

대기업 중견기업 시장
약 1500만 명

SOM

목표 시장 점유율 : 5%

대기업 및 중견기업 수 : 3만개

평균 직원 수 15명

타겟 시장
75만 명

산업재해 예방

한 건설업체가 EAP 도입 후 1년간 산재 발생이 35% 감소하였다. 정신건강 관리 및 우울증 예방은 산재 발생을 감소하는 효과를 보인다는 의미이다. Mood Echo를 통한 근로자의 정신건강 관리를 통해 적은 비용으로 사고 발생률을 감소시킬 수 있다.

생산성 증가

근로자의 정신건강 관리는 집중력과 업무 효율성에 긍정적인 영향을 끼친다. 이를 통해 매출증대 및 기업 성장을 만들어 낼 수 있다.

비용절감

Mood Echo를 통한 근로자의 정신건강 위험 예방을 통해서 결근율과 생산성 저하를 통한 비용을 크게 절감할 수 있다.

기업 이미지 제고

근로자의 우울증을 미리 선별하는 프로그램인 Mood Echo 도입은 기업의 사회적 책임(CSR) 활동으로 인식되어 기업 이미지가 제고 될 것이다. 이는 고객과 주주의 신뢰를 강화하고 인재 유치에 긍정적 효과를 미치게 될 것이다.



출처 한국고용정보원

감정노동이 많은 직업 1위

텔레마케터

출처 산업안전보건공단



텔레마케터 대상 조사에서
10명 중 2-3명이 우울증 증세를 보임.
이들을 대상으로 데이터 수집을 진행.

서비스 시작

2년간 데이터 수집을 마친 후

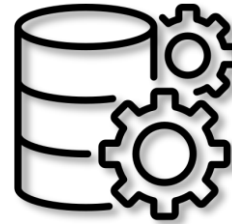
제조/건축 산업 대상으로 서비스 확대



텔레마케터



음성 Data



자사 Database

서비스 발전

B2B 서비스로 쌓은 인지도를 바탕으로

B2C 서비스 시행.



서비스 확대

최종적으로는

AI 상담사를 활용한 헬스케어 어플로 발전



나 너무 힘들엉 헹헹 ;ㅂ;



삐빅! 나약한 인간. 힘내십시오.

최종목표

MoodEcho

1

텔레마케터를 대상으로
서비스 시행하여,
데이터 수집 및 서비스 발전

2

건축/제조 산업을
대상으로 서비스 확대

3

B2B 서비스를
B2C 서비스로 확장

4

AI 상담사를 활용한
심리헬스케어 어플로 발전

문제점

장비 문제



한국어 음성 데이터의 부재



데이터 셋의 절대적 수 부족



해결방안

창업지원자금 지원 신청



AI HUB에서 데이터셋 지원



텔레마케터의 음성 데이터 수집



참고문헌

- Gratch J, Artstein R, Lucas GM, Stratou G, Scherer S, Nazarian A, Wood R, Boberg J, DeVault D, Marsella S, Traum DR. The Distress Analysis Interview Corpus of human and computer interviews. In LREC 2014 May (pp. 3123-3128)
- DeVault, D., Artstein, R., Benn, G., Dey, T., Fast, E., Gainer, A., Georgila, K., Gratch, J., Hartholt, A., Lhommet, M., Lucas, G., Marsella, S., Morbini, F., Nazarian, A., Scherer, S., Stratou, G., Suri, A., Traum, D., Wood, R., Xu, Y., Rizzo, A., and Morency, L.-P. (2014). "SimSensei kiosk: A virtual human interviewer for healthcare decision support". In Proceedings of the 13th International Conference on Autonomous Agents and Multiagent Systems (AAMAS'14), Paris
- Huang X, Wang F, Gao Y, Liao Y, Zhang W, Zhang L, Xu Z. Depression recognition using voice-based pre-training model. Sci Rep. 2024 Jun 3;14(1):12734. doi: 10.1038/s41598-024-63556-0. PMID: 38830969.
- 고용노동부. (2023). 산업 재해 예방과 정신건강의 상관관계.
- 삼성경제연구소. (2022). 근로자 정신건강이 기업 비용에 미치는 영향
- 노동부. (2023). 근로자의 정신건강과 생산성의 관계.
- 한국보건산업진흥원. (2023). 디지털 정신건강관리 솔루션의 활용 현황과 전망.
- 최윤정. (2024). 2024 산업안전기사.
- 노춘호, 신태현. (2016). 철도기관사의 사고와 신체적·심리적 우울, 인지실패의 인과관계. 한국산업안전보건공단.



Thank you

Mood Maker

Mood
Echo

유비온 빅데이터를 활용한
디지털 마케팅 최종프로젝트

부록

Dataset	Accuracy	Depression Rate (0)	Prediction Rate (0)	Depression Rate (1)	Prediction Rate (1)
korean-train	0.968253968	133	137	56	52
korean-test	0.639175258	59	80	38	17
base-960h-train	0.952380952	133	132	56	57
base-960h-test	0.81443299	59	77	38	20